

Modeling, Testbench Design, and Verification of the Executable Simulink Testbench for the 64 x 8 PCC-SVD Massive MIMO-OFDM Link-Level Simulator

Md Moklesur Rahman

Independent Researcher, Oulu, Finland | moklesur.eee@gmail.com | github.com/dipucwc

Abstract

This report presents the system modeling, testbench design, execution control, verification methodology, and executed results of the executable Simulink testbench of the 64 x 8 massive MIMO-OFDM link-level simulator with pilot-compatibility-constrained SVD (PCC-SVD) precoding. The mathematical model is stated in Equations (1)-(4) of this report, the per-resource-element transmission model, the pilot-reconstruction NMSE η_{PR} , the SNR-adaptive compatibility limit, and the feasible-set selection rule, with their complete derivation in the companion journal manuscript [1]. The testbench executes the verified MATLAB numerical engine from an active Level-2 MATLAB S-function and reports a six-element status word on displays and workspace logs. Every quoted value is from executed runs: nine of nine verification gates pass in plain MATLAB and inside the executing model; the deterministic 10 dB smoke test reports BER $3.18287e-3$, NMSE $2.42407e-2$, RMS EVM 0.151627, capacity 28.0944 bit/s/Hz, and a selected bundle of 48 subcarriers; the complete in-model analysis campaign finishes with the final 20 dB PCC-SVD values BER $1.20563e-5$, NMSE $3.5484e-3$, and mean bundle 144, identical to the scripted campaign at the numerical-engine interface; and all twenty-eight MATLAB-Python shared-replay-vector checks pass. The fixed-frame six-scenario sweep is tabulated and analyzed, and the independent MATLAB and Python campaigns are compared at 10 dB.

Keywords: Massive MIMO, OFDM, PCC-SVD, wideband SVD, DM-RS, Simulink testbench, Level-2 S-function, Procrustes alignment, verification gates, MMSE, cross-verification, reproducibility

1. Introduction

This section motivates the executable testbench, states its objectives, and outlines the report. Section 1.1 presents the technical background and motivation, Section 1.2 states the objectives and contributions, and Section 1.3 summarizes the organization of the remaining sections.

1.1 Background and motivation

Frequency-selective massive MIMO precoding faces a structural conflict that the companion manuscript formulates and resolves: per-subcarrier singular-value decomposition maximizes the captured channel subspace but produces effective channels that a comb-based DM-RS estimator cannot reconstruct, while a single wideband basis is perfectly estimator-compatible but sacrifices subspace capture. PCC-SVD resolves the conflict by selecting, per frame, among aligned SVD candidates of bundle sizes 144, 48, and 1 under an explicit SNR-adaptive pilot-reconstruction budget. Results of this kind are only as credible as their verification, and the executable Simulink testbench documented here exists for that purpose: it makes the verified numerical chain runnable inside a model-based environment, re-executes the complete gate suite on every simulation, runs the full campaign from inside the model, and exposes the health and operating point of the chain on a standardized status interface.

1.2 Objectives and contributions

The testbench pursues four objectives. First, execute the verified MATLAB numerical engine from a native Simulink block without duplicating a single numerical operation, so the model can never diverge from the code that produced the published results. Second, embed the complete deterministic verification suite in the model itself, so a simulation that starts is a simulation whose chain has already been proven. Third, expose a compact standardized interface, a six-element status word carrying execution status, passed-check count, total-check count, BER, effective-channel NMSE, and selected bundle size. Fourth, connect the MATLAB engine to the independent Python publication implementation through deterministic shared replay vectors, so cross-language agreement is itself a verified, executed property. This report documents that design, the mathematical model it executes, and its complete executed evidence.

1.3 Report organization

Section 2 states the physical-layer mathematical model with the manuscript equation numbering, the power convention, the precoder candidate families with the compatibility formulation and selection rule, and the executed configuration.

Section 3 describes the Simulink testbench architecture with the executed model figure and the block-to-function traceability. Section 4 presents the algorithm workflow and receiver processing, algorithm by algorithm. Section 5 explains the programmatic model construction and instrumentation. Section 6 defines the verification methodology with the gate table. Section 7 reports the executed results with technical analysis, including the in-model campaign and the independent MATLAB-Python comparison. Section 8 gives the reproduction procedure and project contents, Section 9 states limitations and concludes, and Section 10 lists the references.

2. Physical-Layer Mathematical Model and Simulation Conventions

This section defines the mathematical model and the conventions on which every executed result relies. Section 2.1 states the per-resource-element transmission model with the manuscript equation numbering, Section 2.2 derives the power normalization and the total-transmit-SNR convention, Section 2.3 presents the compatibility formulation with the SNR-adaptive selection rule, and Section 2.4 records the executed configuration.

2.1 Per-resource-element MIMO-OFDM model

The simulator implements a downlink data channel on a compact OFDM grid of 256 FFT bins with 144 active subcarriers over one 14-symbol slot at the 30 kHz numerology, from 64 transmit antennas to 8 receive antennas with $L = 4$ spatial layers of Gray-mapped, unit-average-power 16-QAM. Following the array definitions of [1, Eqs. (1)-(7)], the received signal on active subcarrier k is

$$y[k] = H[k] W[k] s[k] + n[k] \quad (1)$$

where $H[k]$ is the 8×64 physical channel of a five-tap TDL profile with delays (0, 2, 5, 9, 14) samples and relative powers (0, -2.2, -4, -6, -8.2) dB per the TDL frequency response of [1, Eq. (8)], $W[k]$ is the 64×4 precoder of the active strategy, $s[k]$ holds the four layer symbols, and $n[k]$ is circular complex AWGN. The receiver observes and equalizes the effective channel $G[k] = H[k] W[k]$ of dimension 8×4 per subcarrier. OFDM symbols 4 and 11 in MATLAB one-based indexing carry the DM-RS-like reference: each layer occupies one of four non-colliding frequency offsets with comb spacing four following the DM-RS principle of [2], and the selected $W[k]$ is applied to both data and reference layer vectors, so the reference experiences the same spatial transmission as its data.

2.2 Power normalization and total-transmit-SNR convention

All precoders are semi-unitary, so the total transmit energy per resource element equals the layer count L , and the complex noise variance per receive antenna follows the fixed convention derived in [1, Eq. (20)], $\sigma^2 = L / \rho$ with ρ the linear total transmit SNR. Because the convention ties the noise to the transmitted energy rather than to any per-strategy received power, every scenario difference is attributable to the precoding strategy and its estimator interaction alone. The deterministic seed and the matched-realization discipline extend the same attribution across the Monte Carlo dimension: all scenarios at a frame index see identical channels, payloads, and noise.

2.3 Compatibility formulation and the adaptive selection rule

The candidate families are the wideband covariance-SVD basis of [1, Eqs. (9)-(10)], computed by Hermitian eigendecomposition of the band-averaged covariance [4]; the fixed 48-subcarrier subband and per-subcarrier variants of the same construction; and their unitary Procrustes alignment per [1, Eqs. (11)-(12)], which rotates each basis toward its frequency predecessor by the compact-SVD solution of the orthogonal Procrustes problem [3]. The compatibility of a candidate with the implemented estimator is measured without noise: the reconstruction operator of [1, Eq. (25)] forms $G_{\text{rec}}[k]$ by applying the layer-specific pilot sampling operator S_P and the exact receiver interpolation operator I_P to the noise-free G , and Equation (2) defines the pilot-reconstruction NMSE

$$\eta_{\text{PR}}(b) = \sum_k \| G[k] - G_{\text{rec}}[k] \|^2_F / \sum_k \| G[k] \|^2_F \quad (2)$$

which contains interpolation error but no AWGN and therefore isolates the structural mismatch between the effective-channel frequency variation and the configured pilot process. The subspace-capture ratio η_{SC} , defined in [1, Eq. (27)], measures the fraction of dominant channel-subspace energy the candidate retains. The allowed reconstruction error is SNR adaptive per Equation (3),

$$\epsilon_{\text{PR}}(\rho) = \min(\epsilon_{\text{max}}, \epsilon_{\text{inf}} + \beta_{\text{PR}} / \rho) \quad (3)$$

with $\epsilon_{\text{max}} = 0.008$, $\epsilon_{\text{inf}} = 0.002$, and $\beta_{\text{PR}} = 0.03$; at 10 dB the budget evaluates to 0.005 and at 20 dB to 0.0023, so the constraint tightens exactly where the interpolation floor becomes visible. Equation (4) states the piecewise selection over the feasible set $F(\rho) = \{ b : \eta_{\text{PR}}(b) \leq \epsilon_{\text{PR}}(\rho) \}$, following [1, Eqs. (29)-(30)]:

$$b^* = \arg \max \text{ over } b \text{ in } F(\rho) \text{ of } \eta_{SC}(b) \text{ if } F(\rho) \text{ is non-empty; } b^* = \arg \min \text{ over } b \text{ of } \eta_{PR}(b) \text{ otherwise} \quad (4)$$

The feasible candidate with the largest captured subspace energy is selected when the budget admits at least one candidate, and the minimum- η_{PR} candidate otherwise, so a severe channel realization degrades gracefully instead of failing. The equalizer-consistency error defined in [1] completes the diagnostic set by re-applying the detector to its own model and measuring the residual, which is at machine precision for ideal-CSI zero-forcing.

2.4 Executed configuration

Table 1 reproduces the executed configuration recorded in `simulation_configuration.txt`; the dimensions correspond directly to the arrays of [1, Eqs. (1)-(7)], and the TDL, pilot, candidate-bundle, and adaptive-limit entries determine the behavior of Equations (2)-(4) and of the estimator of Section 4.2. The fixed-frame setting of six frames per SNR point defines the statistical resolution of the testbench sweep, discussed in Section 7.4.

Table 1. Executed configuration; parameter roles follow Table 2 of the manuscript.

Parameter	Value	Parameter	Value
FFT size / active subcarriers	256 / 144	Antennas nTx x nRx / layers	64 x 8 / 4
OFDM symbols per slot	14 at 30 kHz SCS	Modulation	16-QAM, 4 bits per symbol
DM-RS symbols / comb spacing	4 and 11 / 4	Channel model / seed	5-tap TDL / 7
TDL delays (samples)	0, 2, 5, 9, 14	TDL powers (dB)	0, -2.2, -4, -6, -8.2
Equalizer	Gain-corrected MMSE (ZF available)	Frames per SNR point	6
SNR grid (dB)	-5, 0, 5, 10, 15, 20	PCC candidate bundles	144, 48, 1
epsilon_max / epsilon_inf / beta_PR	0.008 / 0.002 / 0.03	Budget at 10 / 20 dB	5.000e-3 / 2.300e-3

3. Simulink Testbench Architecture

This section describes the generated Simulink model. Section 3.1 presents the graphical organization of the executed model with its figure, Section 3.2 establishes the block-to-function traceability together with the equation-to-stage mapping, and Section 3.3 documents the fixed-step execution and logging configuration.

3.1 Graphical system organization

Figure 1 shows the executed model `Wideband_SVD_DMRS_Executable_Testbench.slx`. The active execution path holds, in signal order, a unit-step Run Trigger that fires the engine once at time zero, the Level-2 MATLAB S-function block `simulink_sfunction`, a Unit Delay Status Register that holds the six-element result vector between steps, a six-output demultiplexer, six Display blocks labeled Status, Checks Passed, Total Checks, BER, NMSE, and Selected Bundle, a To Workspace log in structure-with-time format under `simulink_status_log`, and a Scope with an output port. The lower region is the Implemented Algorithm Workflow subsystem, which documents the processing sequence of the engine as a labeled block chain: configuration, frame and DM-RS generation, TDL channel generation, SVD candidate construction, PCC-SVD selection, digital precoding, MIMO propagation with AWGN, DM-RS least-squares estimation, CSI selection, ZF or MMSE equalization, and demapping with metric calculation. The workflow region is documentation by construction: it is not a second native block-by-block implementation of the 64 x 8 matrix operations, and every numerical calculation remains inside the verified engine.

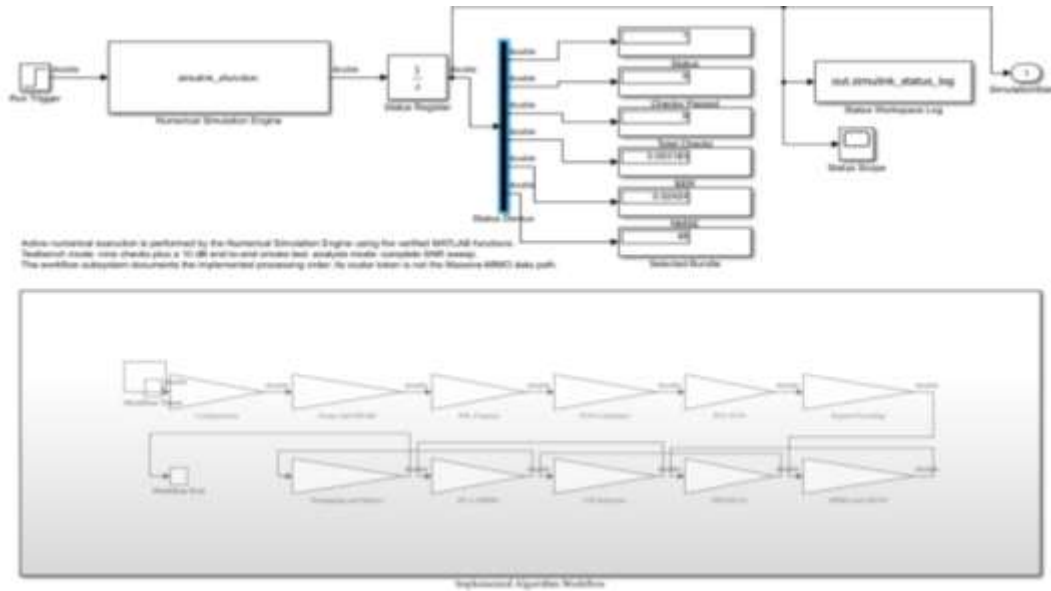


Figure 1. Executed Simulink Level-2 MATLAB S-function orchestration testbench with verification and campaign outputs.

Figure 1 reproduces the executed model of the manuscript. The Run Trigger activates the S-function, whose six-element output feeds the Status Register, the demultiplexer, and the six displays, with workspace logging and the scope on the same word. The displayed values of a testbench-mode run report all checks passed together with the smoke-test operating point of Section 7.1, and the lower workflow subsystem documents the eleven processing stages of the numerical engine.

3.2 Block-to-function traceability

Each element of the model traces to exactly one source artifact. The S-function block executes `simulink_sfunction.m`; the S-function delegates all numerical work to `simulink_runner.m`; and the runner calls the verified engine `massive_mimo_ofdm_simulation.m` together with the gate suite of `matlab_testbench.m`. Inside the engine, the manuscript stages carry the names `buildFrame` and `applyPrecoder` for layer mapping and digital precoding, `generateTdlChannel` for the physical channel, `covarianceSvdCandidate` for the candidate bank, `alignSvdBases` for Procrustes alignment, `precoderChannelEstimatorCompatibilitySvd` for the PCC-SVD selection of Equations (2)-(4), `trueEffectiveChannel` for effective-channel formation, `estimateEffectiveChannelLs` for the DM-RS LS estimator, and `equalizeLayers` for ZF and gain-corrected MMSE detection. Table 2 states the mapping between the mathematical operations, the manuscript equations and algorithms [1], and these MATLAB stages, so the report, the algorithms, and the code cannot become independent descriptions of different systems.

Table 2. Mapping between mathematical operations, equations, algorithms, and MATLAB stages.

Technical operation	Manuscript equations / algorithm [1]	MATLAB stage
Layer mapping and digital precoding	(1)-(3), Algorithms 1-2	<code>buildFrame</code> ; <code>applyPrecoder</code>
TDL physical channel	(8)	<code>generateTdlChannel</code>
Covariance-SVD candidate bank	(9)-(10), Algorithms 1-2	<code>covarianceSvdCandidate</code>
Unitary Procrustes alignment	(11)-(12), Algorithm 3	<code>alignSvdBases</code>
PCC-SVD selection	(25)-(30), Algorithm 4	<code>precoderChannelEstimatorCompatibilitySvd</code>
Effective-channel formation	(6)-(7)	<code>trueEffectiveChannel</code>
DM-RS LS estimation	(13)-(16), Algorithm 5	<code>estimateEffectiveChannelLs</code>
ZF / gain-corrected MMSE	(17)-(19)	<code>equalizeLayers</code>
Metrics and confidence reporting	(21)-(24), (31), (33)-(35), Algorithm 6	metric functions / CSV writers

3.3 Fixed-step execution and logging

The model runs a fixed-step discrete solver with step one from time zero to one, so the S-function start callback executes the numerical work exactly once and the output callback streams the stored status word for the remainder of the run. Signal logging, workspace output return, and port data-type display are enabled at the model level; Fast Restart and block reduction are disabled so the executed diagram always matches the displayed diagram. The status word is

therefore available three ways after every run: on the six displays, in the returned summary structure of the runner, and in the logged `simulink_status_log` structure.

4. Algorithm Workflow and Receiver Processing

This section presents the algorithm workflow that the testbench executes. Section 4.1 traces the one-frame execution procedure with the workflow figure and the array contracts, Section 4.2 details the DM-RS-based effective-channel estimation, Section 4.3 details the linear detection, and Section 4.4 defines the performance and diagnostic metrics.

4.1 One-frame execution procedure

Figure 2 shows the complete per-frame algorithm the engine executes and the workflow subsystem documents. The manuscript array contracts govern every stage: the layer grid is $14 \times 144 \times 4$, the physical channel $144 \times 8 \times 64$, the frequency-indexed precoder $144 \times 64 \times 4$ when frequency dependent, the effective channel $144 \times 8 \times 4$, and the received grid $14 \times 144 \times 8$. Algorithms 1 and 2 of [1] construct the frame and the covariance-SVD candidate bank by Hermitian eigendecomposition [4], Algorithm 3 aligns consecutive bases by the compact-SVD Procrustes solution [3], Algorithm 4 evaluates η_{PR} and η_{SC} for every bundle size and applies the selection rule of Equation (4), Algorithm 5 performs the DM-RS LS estimation, and Algorithm 6 performs detection, metric accumulation, and the Monte Carlo loop.

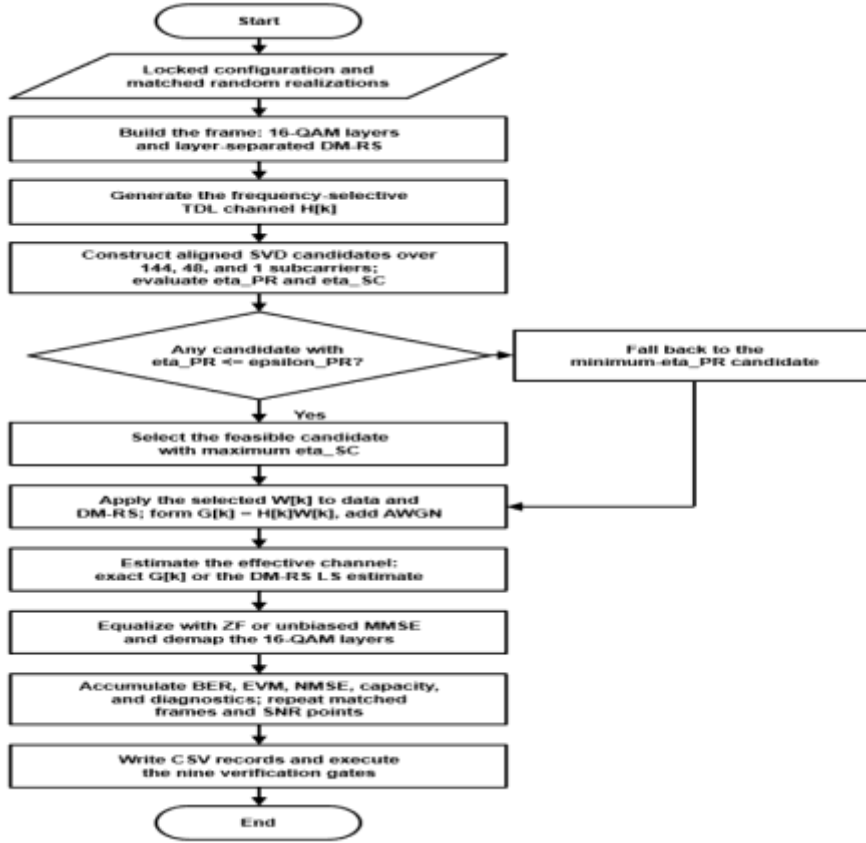


Figure 2. Per-frame algorithm workflow of the PCC-SVD massive MIMO-OFDM chain.

Figure 2 traces one Monte Carlo frame in the execution order of the engine: locked configuration and matched random realizations, frame construction with layer-separated DM-RS, TDL channel generation, construction of the aligned SVD candidates over 144, 48, and 1 subcarriers with evaluation of η_{PR} and η_{SC} , the feasibility decision of Equation (4) with maximum- η_{SC} selection and minimum- η_{PR} fallback rejoining at the precoding stage, precoding of data and DM-RS with noise at the convention of Section 2.2, effective-channel estimation from exact or DM-RS least-squares CSI, equalization and demapping, metric accumulation over matched frames and SNR points, and the final CSV records with the verification gates.

4.2 DM-RS-based effective-channel estimation

The estimated-CSI branch divides each received reference sample by its known unit-modulus QPSK symbol, which is the LS solution for the scalar observation at a layer-separated reference resource element, averages the two DM-RS

symbol observations, and interpolates the real and imaginary components separately across frequency with endpoint holding per [1, Eqs. (13)-(16)] and the reference-signal structure of [2]. The interpolation stage reconstructs coefficients at unsounded subcarriers, and it is exactly this operator pair, sampling S_P and interpolation I_P , that the compatibility metric of Equation (2) applies to the noise-free effective channel: η_{PR} is the normalized error of this receiver applied to each candidate, so the selection rule optimizes against the implemented estimator, not an abstraction of it.

4.3 Linear detection

Detection uses per-subcarrier zero-forcing or MMSE filtering of the selected CSI per [1, Eqs. (17)-(19)], with the MMSE system solved by left matrix division rather than an explicit inverse [4] and followed by the diagonal gain correction that removes the MMSE bias so the 16-QAM decision regions are centered. The corrected MATLAB implementation preserves pilot-vector column orientation and restores resource-element-major, layer-minor symbol ordering before hard demapping, two ordering properties that the verification gates of Section 6 test explicitly. The ideal-CSI branch supplies the exact $G[k]$ and separates the spatial gain available under exact CSI from the performance realizable with the sparse-reference estimator.

4.4 Performance and diagnostic metrics

Per scenario and SNR point the engine records BER, RMS EVM, effective-channel NMSE, and the exact layer capacity of the effective channel [1, Eqs. (21)-(24)] [8], together with the diagnostic set that makes the adaptive mechanism observable: effective-channel smoothness, the pilot-reconstruction NMSE of the selected candidate, the subspace-capture ratio, the equalizer-consistency error, the mean selected bundle size, and the active limit $\epsilon_{PR}(\rho)$. All columns are stored in the per-scenario CSV tables, so the analysis of Section 7 reads the selection behavior directly from the executed records.

5. Programmatic Model Construction and Instrumentation

This section explains how the model is built, instrumented, and kept consistent with its sources. Section 5.1 covers the builder and the model lifecycle scripts, Section 5.2 the Level-2 S-function and its two execution modes, and Section 5.3 the status interface, displays, and logging.

5.1 Builder and model lifecycle

The builder `build_simulink_testbench.m` constructs the model from an empty system on every invocation through the programmatic model-construction interface [7], which makes the SLX file a build artifact rather than a hand-maintained asset: it resolves the local source folder, prints the resolved builder and S-function paths for provenance, removes any older loaded or saved copy, creates the fixed-step discrete model, adds and wires every block programmatically with smart autorouting, constructs the workflow subsystem, and saves the SLX file. The lifecycle scripts keep that property safe: `prepare_simulink_model.m` creates or refreshes the model for the orchestrated run, `force_rebuild_simulink.m` and `reset_and_build_simulink.m` guarantee clean regeneration, `stop_simulation_if_needed.m` stops a running copy and disables Fast Restart before replacement, and `validate_local_path.m` pins execution to the local package so no shadowed function copy can be picked up.

5.2 Level-2 S-function and execution modes

The S-function follows the Level-2 MATLAB S-function interface [6]: it defines one scalar input, one six-element output, and a one-second discrete sample time, and allocates a six-element discrete work vector in its post-propagation callback. All numerical work happens in the simulation-start callback: it reads the requested mode from the base-workspace variable `simulink_execution_mode`, executes the numerical runner, validates the returned status, and stores the packed word. In `testbench` mode the runner executes the nine verification gates and the deterministic 10 dB PCC-SVD smoke test; in `analysis` mode it executes the complete fixed-frame six-scenario MATLAB campaign and returns the final PCC-SVD values. An unsupported mode raises an explicit error, and invoking the S-function outside Simulink prints a usage message directing to `run_simulink_only` or `run_all`.

5.3 Status interface, displays, and logging

The six-element status word standardizes the observable state: overall status, passed-check count, total-check count, BER, effective-channel NMSE, and mean selected bundle size. It drives the six displays for live observation, the workspace log for post-run analysis, and the standardized summary structure whose fields `mode`, `status`, `passCount`, `totalCount`, `finalBer`, `finalNmse`, `finalBundle`, and `message` give scripted callers a fixed contract in both modes; the

runner initializes every field before execution so the S-function always receives a complete structure even on an error path.

6. Verification Methodology

This section defines the verification methodology of the testbench. Section 6.1 presents the nine deterministic gates with the failure modes they target, Section 6.2 presents the deterministic smoke test and the in-model campaign, and Section 6.3 presents the MATLAB and Python shared-vector cross-verification with the executed workflow figure.

6.1 Deterministic verification gates

Nine gates protect the chain, executed identically by the plain-MATLAB internal testbench and, inside the model, by the S-function in testbench mode. Each gate targets a specific failure mode, unfair transmit-power scaling, inconsistent dimensions, mapper ordering errors, incorrect pilot division, detector ordering errors, ineffective basis alignment, or an incomplete fallback rule; passing all gates does not prove global correctness, but failing any gate invalidates the corresponding campaign, and the S-function refuses a passing status in that case. Table 3 lists the gates with their technical purposes and the executed result.

Table 3. Verification gates, failure modes tested, and executed results, following [1, Table 6].

Verification gate	Technical purpose	Result
Wideband semi-unitarity	Verify $W^H W = I_L$ and equal power normalization	PASS
Aligned per-subcarrier semi-unitarity	Verify alignment preserves the selected subspace basis norm	PASS
PCC-SVD semi-unitarity	Verify the selected frequency-indexed precoder remains semi-unitary	PASS
Effective-channel dimensions	Verify $G[k]$ has dimension $K \times N_R \times L$ (144 x 8 x 4)	PASS
16-QAM round trip	Verify mapper and demapper bit and symbol ordering	PASS
Noiseless flat-channel LS	Verify exact pilot-division and interpolation recovery	PASS
Noiseless end-to-end recovery	Verify mapping, equalization, ordering, and demapping	PASS
Alignment reduces pilot error	Verify Procrustes alignment reduces η_{PR}	PASS
PCC-SVD feasibility or fallback	Verify Equation (4) including the fallback branch	PASS

6.2 Smoke test and in-model campaign

The testbench mode closes with a deterministic 10 dB smoke test of the full estimated-CSI PCC-SVD chain whose role is regression sensitivity: its values are reproduced exactly by the deterministic seeds, so any numerical drift anywhere in the chain changes the smoke test before it changes a published curve. The analysis mode raises the same principle to the campaign level: the model itself executes the complete fixed-frame six-scenario sweep through the identical engine paths that the scripted campaign uses, so the in-model final values and the scripted CSV rows must agree exactly, and Section 7.1 confirms that they do.

6.3 MATLAB and Python shared-vector cross-verification

Cross-language verification follows [1, Algorithm 7] with deterministic shared replay vectors: `shared_vectors.mat` with its JSON manifest stores, for each of the four precoder families, matched realizations and reference quantities generated by the Python side, and `verify_shared_vectors.m` confirms seven properties per family in MATLAB, dimensions, effective-channel agreement, semi-unitarity, stored orthogonality, pilot-reconstruction NMSE, subspace capture, and the recorded selected bundle, so numerical-convention regressions can be re-checked without random-number-generator ambiguity. Figure 3 shows the complete executed verification workflow, including the abort path that accepts no result when any gate fails and the investigation loop that returns to the gate stage after any fix.

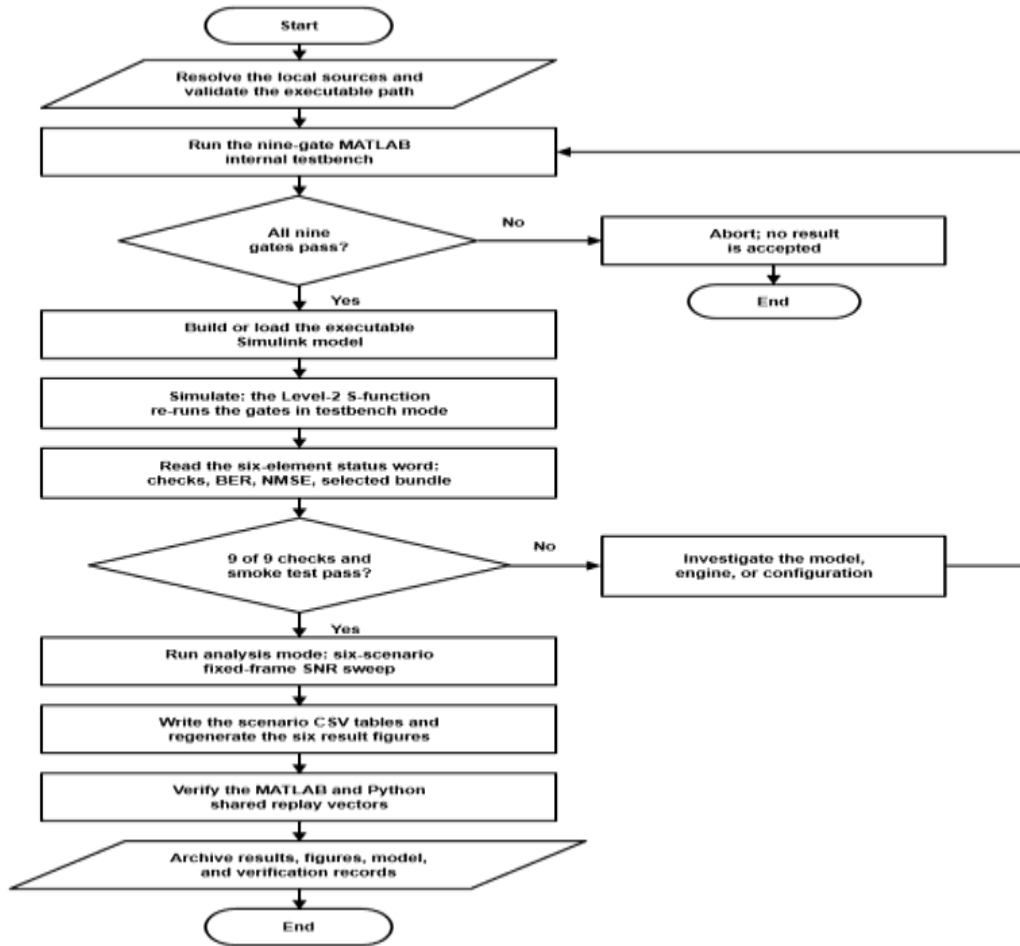


Figure 3. Executable-testbench execution and verification workflow.

Figure 3 traces the executed verification order: source resolution and path validation, the nine-gate plain-MATLAB testbench with its abort path, model construction or loading, the in-model re-execution of the gates with the smoke test through the Level-2 S-function, the six-element status word, the analysis-mode fixed-frame sweep with CSV and figure regeneration, the MATLAB and Python shared-vector verification, and the archival of all records. A failed in-model check routes to investigation and back to the gate stage, so no result is ever accepted over a broken chain.

7. Executed Results and Technical Analysis

This section reports the executed results with technical analysis. Section 7.1 presents the in-model executed results and the MATLAB-Simulink agreement, Section 7.2 presents the complete fixed-frame metric results with the regenerated BER figure, Section 7.3 interprets the observed trends, and Section 7.4 presents the independent cross-implementation comparison and the relation to the publication campaign.

7.1 In-model executed results and MATLAB-Simulink agreement

Every value in this section is from executed runs recorded in the package and the manuscript. The plain-MATLAB internal testbench records nine of nine gates passing. The executing Simulink model records the identical nine checks passing, and its deterministic 10 dB smoke test reports BER $3.18287\text{e-}3$, effective-channel NMSE $2.42407\text{e-}2$, RMS EVM 0.151627, capacity 28.0944 bit/s/Hz, a selected-candidate pilot-reconstruction error of $3.809\text{e-}3$ under the $5.000\text{e-}3$ budget, and a selected bundle of 48 subcarriers. The complete in-model analysis campaign was also executed: it finished with all nine checks passing and reported the final 20 dB PCC-SVD values BER $1.20563\text{e-}5$, NMSE $3.5484\text{e-}3$, and mean bundle 144, which are digit-for-digit identical to the 20 dB row of the scripted campaign CSV. The direct MATLAB campaign and the Simulink wrapper therefore agree exactly at the numerical-engine interface, which is the designed consequence of the orchestration architecture: the model executes the same code, so agreement is structural rather than statistical. The shared-vector verification records all twenty-eight cross-language checks passing across the four precoder families.

7.2 Complete fixed-frame metric results

Table 4 summarizes the estimated-CSI and ideal-CSI bit error rates of the analysis-mode sweep at four operating points together with the mean selected bundle of the adaptive method; the full per-scenario tables with EVM, NMSE, capacity, and all diagnostic columns are stored in the results_matlab folder. Figures 4 through 9 reproduce the six figures regenerated by the executed sweep and indexed in plot_index.csv: the estimated-CSI BER, the DM-RS LS estimation NMSE, the equalized-symbol RMS EVM, the ideal-CSI against realizable estimated-CSI BER comparison, the exact effective-channel capacity, and the identity-precoded ZF and MMSE reference.

Table 4. Executed fixed-frame BER of the analysis-mode sweep at four operating points.

Scenario	BER -5 dB	BER 0 dB	BER 10 dB	BER 20 dB	Mean bundle -5 / 20 dB
Wideband SVD, LS, MMSE	2.936e-1	1.627e-1	6.119e-3	1.206e-5	144 / 144
Aligned per-subcarrier SVD, LS, MMSE	1.943e-1	7.574e-2	2.532e-4	1.085e-4	1 / 1
Unaligned per-subcarrier SVD, LS, MMSE	3.288e-1	2.602e-1	2.316e-1	2.299e-1	1 / 1
PCC-SVD, LS, MMSE	2.036e-1	8.362e-2	3.370e-3	1.206e-5	8.83 / 144
Wideband SVD, ideal CSI, MMSE	1.775e-1	7.510e-2	2.411e-4	0	144 / 144
Per-subcarrier SVD, ideal CSI, MMSE	9.624e-2	1.773e-2	0	0	1 / 1

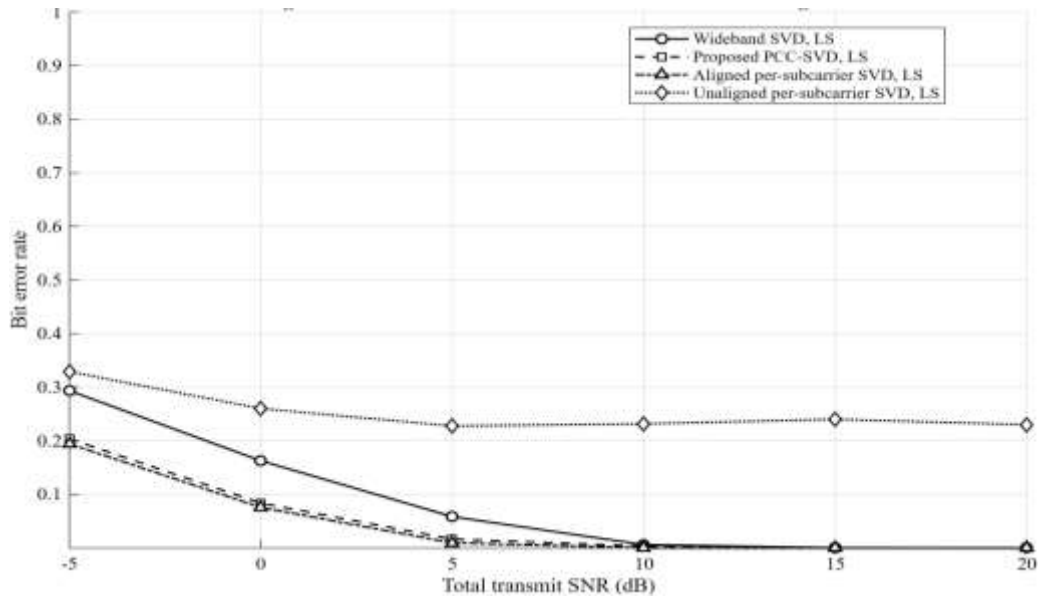


Figure 4. Executed estimated-CSI BER of the evaluated SVD strategies from the analysis-mode sweep.

Figure 4 reproduces the main BER figure regenerated by the executed sweep, stored in the package as plot_01_ber_main.png with its entry in the plot index. The curves display the four estimated-CSI scenarios of Table 4 and show the adaptive method following the aligned per-subcarrier curve at low SNR and joining the wideband curve at high SNR, with the unaligned failure floor separated from all working strategies by orders of magnitude.

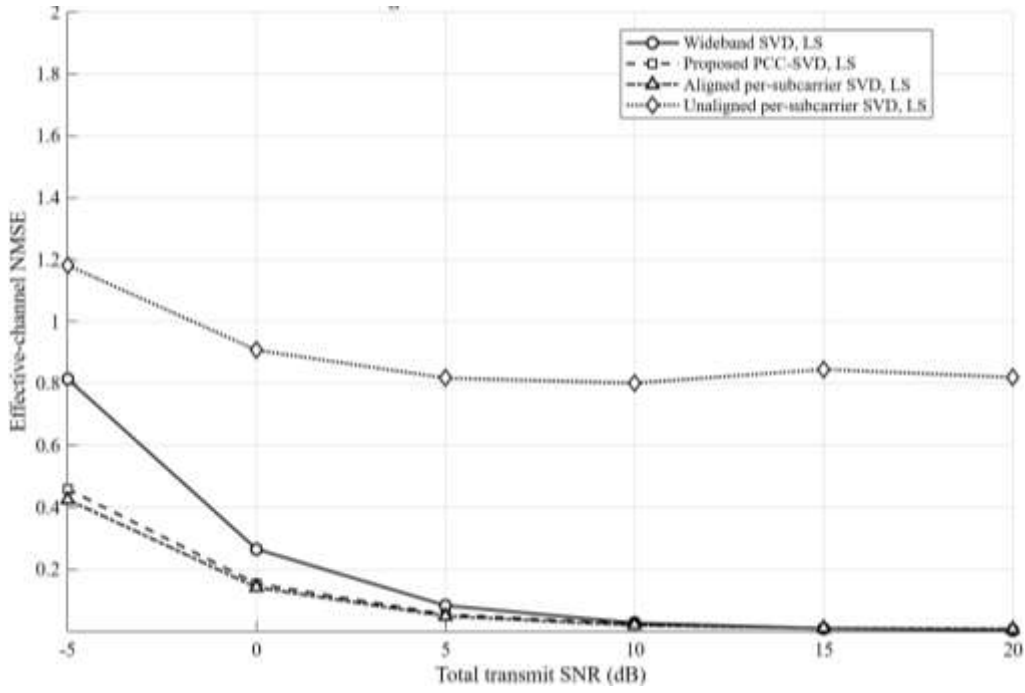


Figure 5. Executed DM-RS LS effective-channel estimation NMSE of the evaluated strategies.

Figure 5 reproduces the estimation-NMSE figure of the sweep, stored as plot_02_nmse_main.png. The unaligned per-subcarrier curve holds an SNR-independent NMSE near 0.8 because its effective channel decorrelates between pilot positions, while the aligned strategies decrease with SNR toward their interpolation-limited values; at 20 dB the wideband and PCC-SVD selections reach $3.548\text{e-}3$ and the aligned per-subcarrier basis $7.459\text{e-}3$, which orders the strategies exactly as the pilot-reconstruction metric η_{PR} of Equation (2) predicts.

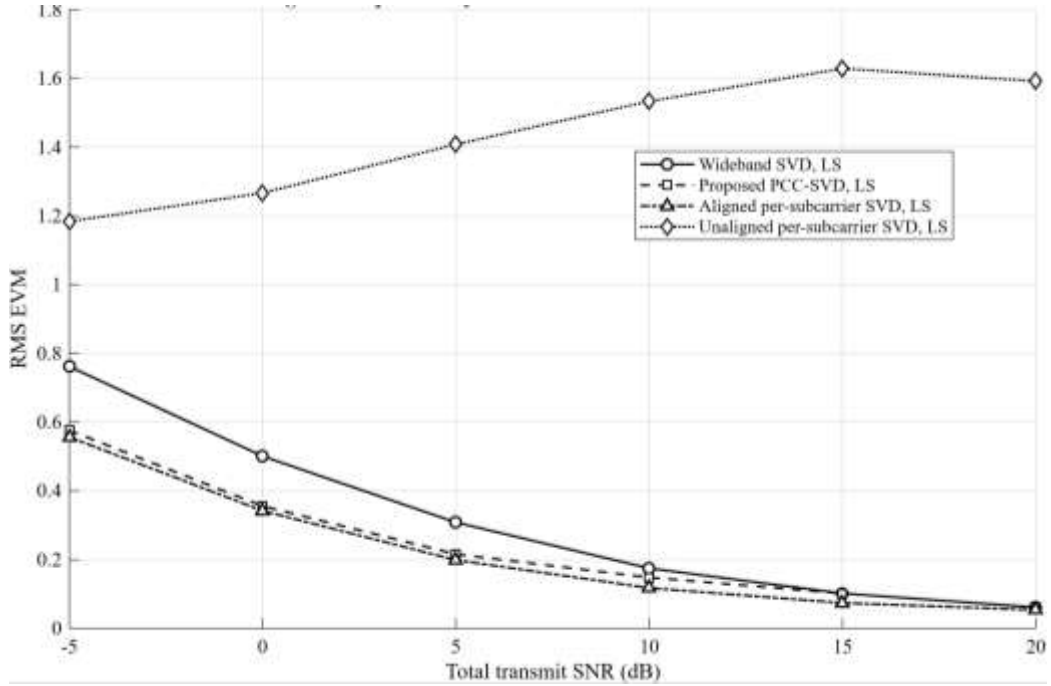


Figure 6. Executed equalized-symbol RMS EVM with estimated CSI.

Figure 6 reproduces the RMS EVM figure of the sweep, stored as plot_03_evm_main.png. The EVM curves mirror the BER ordering of Figure 4: the unaligned basis saturates above one hundred percent, and at 20 dB the PCC-SVD and wideband selections reach 6.11 percent while the aligned per-subcarrier basis reaches 5.35 percent, whose lower residual reflects its larger captured subspace at the cost of the estimation floor visible in Figure 5.

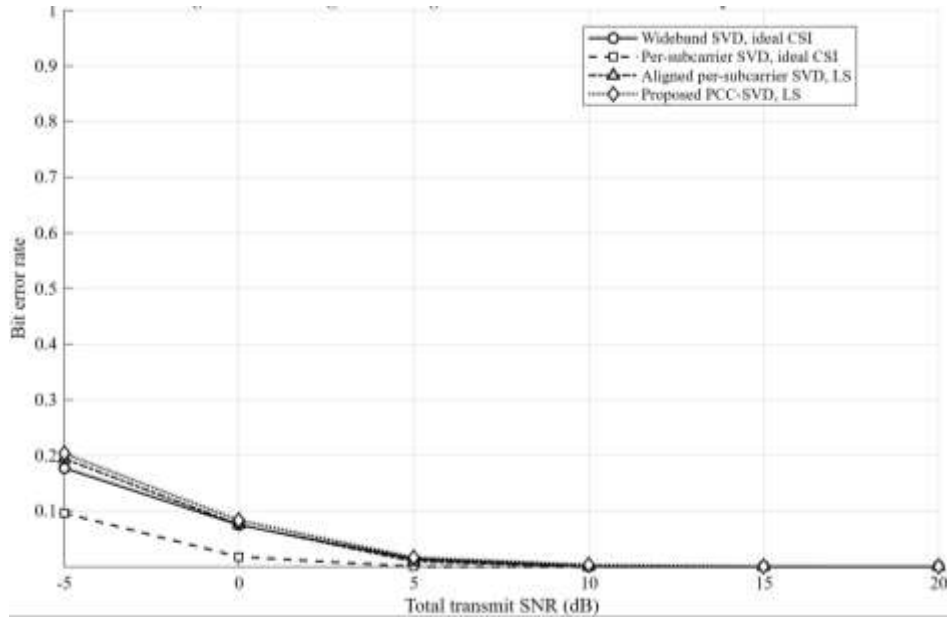


Figure 7. Executed ideal-CSI and realizable estimated-CSI BER comparison.

Figure 7 reproduces the ideal-against-estimated comparison, stored as `plot_04_ideal_ber.png`. The ideal-CSI branches quantify the spatial gain available under exact knowledge of $G[k]$, with the per-subcarrier basis error-free from 10 dB and the wideband basis from 15 dB in the fixed-frame record, and the estimated-CSI branches show what the sparse-reference estimator realizes; the separation between the two branch families is the price of comb-based estimation that the compatibility formulation of Section 2.3 manages explicitly.

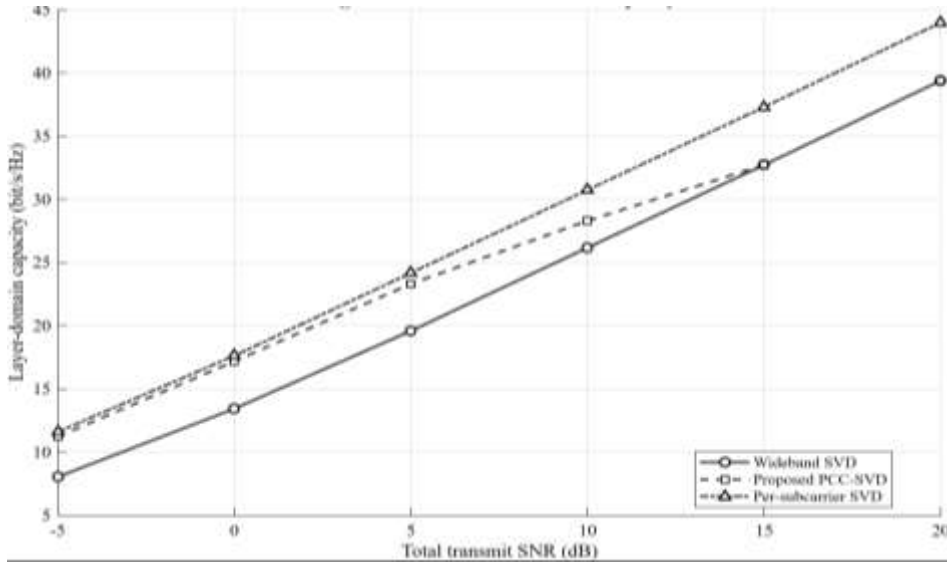


Figure 8. Executed exact effective-channel capacity comparison.

Figure 8 reproduces the capacity figure of the sweep, stored as `plot_05_capacity.png`. The per-subcarrier basis attains the largest exact capacity, 43.99 bit/s/Hz at 20 dB against 39.4 bit/s/Hz for the wideband and PCC-SVD selections, which quantifies the subspace-capture advantage that the η_{SC} metric measures; the adaptive method deliberately concedes this margin at high SNR because Equation (3) tightens the reconstruction budget where the estimation floor would otherwise dominate the error rate.

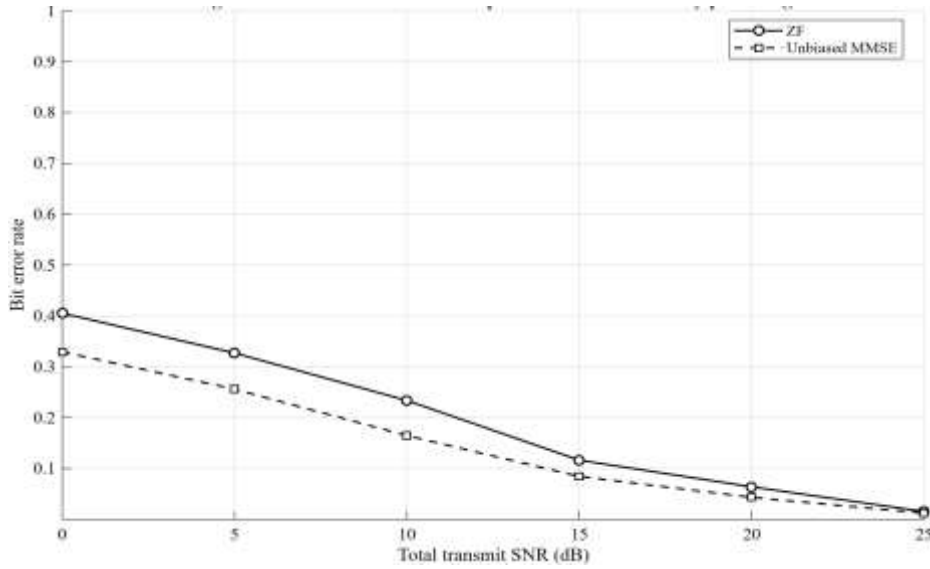


Figure 9. Executed ZF and unbiased-MMSE identity-precoded reference.

Figure 9 reproduces the identity-precoded 4 x 4 reference, stored as plot_06_zf_mmse.png. Without transmit precoding the unbiased MMSE detector outperforms zero-forcing across the grid, $4.31\text{e-}2$ against the ZF curve at 20 dB in the fixed-frame record, and the ZF branch confirms detector self-consistency with an equalizer-consistency error at machine precision; the reference isolates pure detector behavior from every precoding and estimation effect studied in the main scenarios.

7.3 Interpretation of the trends

The executed sweep reproduces every qualitative mechanism the manuscript establishes. The unaligned per-subcarrier SVD exhibits the SNR-independent failure floor near 0.23 that motivates alignment: its effective channel decorrelates between pilot positions, so additional SNR cannot repair the estimate. The aligned per-subcarrier SVD is strongest at low SNR, where AWGN dominates the interpolation error, but retains a floor of about $1.1\text{e-}4$ at 20 dB where the interpolation error dominates. The wideband basis behaves in the opposite order, weak at low SNR and strongest at 20 dB with $1.206\text{e-}5$. PCC-SVD interpolates between the two exactly as Equations (3)-(4) dictate: at -5 and 0 dB the loose 0.008 budget admits fine bundles and the mean selection of 8.83 subcarriers tracks the aligned per-subcarrier error rate; as the budget tightens through $5.000\text{e-}3$ at 10 dB the mean bundle rises through 16.7 and 64.3; and at 15 and 20 dB the $2.3\text{e-}3$ budget forces the wideband candidate, where the adaptive curve matches the wideband curve digit for digit because the selections coincide. The recorded selection histogram over all thirty-six frames, sixteen per-subcarrier, six 48-subcarrier, and fourteen wideband selections, is the direct executed evidence of this transition, and the equalizer-consistency error of the ideal-CSI zero-forcing baseline sits at machine precision, near $1\text{e-}30$, confirming detector self-consistency.

7.4 Cross-implementation comparison and the publication campaign

Table 5 compares the executed MATLAB campaign with the independent Python publication campaign at 10 dB, following [1, Table 4]. The implementations use the same equations, dimensions, channel statistics, pilot positions, interpolation rule, and SNR normalization, but their Monte Carlo random streams are independent because NumPy and MATLAB do not produce identical arrays from the same integer seed; agreement in metric scale and scenario ordering is therefore the appropriate comparison, and sample-identical equality is not claimed. The statistical claims of the project rest on the high-statistics Python campaign with adaptive stopping and exact Clopper-Pearson intervals [5], in which PCC-SVD reaches $2.978\text{e-}3$ at 10 dB against $7.069\text{e-}3$ for the wideband baseline and $5.425\text{e-}6$ at 20 dB with a ninety-five percent interval of $1.991\text{e-}6$ to $1.181\text{e-}5$; the fixed-frame testbench values are consistent with those curves at every point where six frames of resolution permit comparison, which is exactly the division of labor the two-artifact design intends.

Table 5. Independent MATLAB and Python campaign results at 10 dB, following [1, Table 4].

Scenario	MATLAB BER	Python BER	MATLAB NMSE	Python NMSE
Wideband SVD, LS	6.119e-3	7.069e-3	2.696e-2	2.742e-2
PCC-SVD, LS	3.370e-3	2.978e-3	2.322e-2	2.325e-2
Aligned per-subcarrier SVD, LS	2.532e-4	4.145e-4	1.927e-2	1.962e-2
Unaligned per-subcarrier SVD, LS	2.316e-1	1.991e-1	8.017e-1	6.651e-1

8. Reproduction Procedure and Project Contents

This section gives the reproduction procedure and documents the project contents. Section 8.1 lists the executable commands that regenerate every result of this report, and Section 8.2 documents the file organization of the package.

8.1 Reproduction workflow

The complete verification executes with one command from the package folder: `run_all` resolves the local dependencies, executes the scripted curves with figure generation, prepares the model, runs the executable Simulink verification, and prints the completion summary with result-figure restoration. The Simulink layer alone executes with `summary = run_simulink_only('testbench')` for the gates and the smoke test, or `run_simulink_only('analysis')` for the complete in-model campaign; both leave the status word on the model displays and in `simulink_status_log`. A clean regeneration of the model uses `reset_and_build_simulink` or `force_rebuild_simulink`, and `checks = verify_shared_vectors` executes the twenty-eight cross-language checks against `shared_vectors.mat`. Every table and figure of Section 7 regenerates from these commands with the deterministic seed of Table 1.

8.2 Project file organization

The package is organized around three roles. The numerical engine and its verification are `massive_mimo_ofdm_simulation.m` and `matlab_testbench.m`, with the campaign drivers `run_main.m`, `run_complete_simulation.m`, `run_high_statistics_analysis.m`, and the orchestrator `run_all.m`. The Simulink layer comprises `build_simulink_testbench.m`, `simulink_sfunction.m`, `simulink_runner.m`, `run_simulink_only.m`, `prepare_simulink_model.m`, `reset_and_build_simulink.m`, `force_rebuild_simulink.m`, `stop_simulation_if_needed.m`, `validate_local_path.m`, and the generated model `Wideband_SVD_DMRS_Executable_Testbench.slx`. Cross-language verification uses `shared_vectors.mat`, `shared_vectors_manifest.json`, and `verify_shared_vectors.m`. The executed records live in `results_matlab` with the gate record, the configuration log, and the per-scenario CSV tables including the selection histogram; the regenerated figures live in `figures_matlab` with the plot index; the high-statistics records live in `high_statistics_results_matlab` and `high_statistics_figures_matlab`; and the figure managers `focus_result_figures.m` and `hold_result_figures.m` restore the result windows after a run.

9. Limitations, Future Work, and Conclusion

This section states the limits of the testbench so that its results are interpreted only within the executed assumptions. The model represents a single-cell, single-user digital-baseband downlink with uncoded hard-decision 16-QAM and ideal timing and frequency synchronization; impairment injection and synchronization recovery are outside the scope of this bench. The Simulink layer follows the orchestration pattern deliberately: it executes the verified engine from a single active S-function and documents the processing order in the workflow subsystem, so it verifies numerical integrity, standardizes observation, and runs the complete campaign in the model environment, but it is not a native block-by-block implementation of the 64×8 matrix operations; a natural future step is a hierarchical block-level variant built over the same engine through thin per-stage wrappers. The fixed-frame sweep resolves six frames per SNR point, which bounds its statistical resolution as discussed in Section 7.4; the statistical evidence of the project lives in the high-statistics campaign of the manuscript and its archived records.

9.1 Conclusion

The executable Simulink testbench turns the verified PCC-SVD simulation chain into an observable, self-checking, campaign-capable instrument. Its architecture guarantees by construction that the model can never diverge from the code that produced the published results; its embedded gate suite guarantees that no simulation produces results over a broken chain; and its executed records, nine of nine gates in MATLAB, nine of nine checks inside the model with a

fully quantified smoke test, an in-model campaign whose final values match the scripted campaign digit for digit, twenty-eight of twenty-eight shared-vector checks across languages, and a fixed-frame sweep that reproduces every mechanism of Equations (2)-(4), demonstrate that the design achieves its purpose. Together with the journal manuscript and the archived high-statistics campaign, the bench completes a reproducible, verifiable model-based engineering workflow from mathematical model to published result.

10. References

- [1] M. M. Rahman, "Performance Analysis of Wideband SVD Precoding and DM-RS-Based Effective-Channel Estimation in 64 x 8 Massive MIMO-OFDM Systems," accompanying journal manuscript of this project, 2026.
- [2] 3GPP TS 38.211, "NR; Physical channels and modulation," Release 17.
- [3] P. H. Schonemann, "A generalized solution of the orthogonal Procrustes problem," *Psychometrika*, vol. 31, no. 1, pp. 1-10, 1966.
- [4] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.
- [5] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404-413, 1934.
- [6] MathWorks, "Write Level-2 MATLAB S-Functions," Simulink documentation, MathWorks, Natick, MA.
- [7] MathWorks, "Build and Edit a Model Programmatically," Simulink documentation, MathWorks, Natick, MA.
- [8] D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*, Cambridge University Press, 2005.